ARoT: Augmented Reality over IoT for Real-Time Industrial Machine Monitoring

G B Panduranga
Department of Master of Computer Applications
R V College of Engineering
Bengaluru , India
pandurangagb.mca24@rvce.edu.in

Prof. Savita Sheelavant
Assistant Professor
Department of Master of Computer Applications
R V College of Engineering
Bengaluru , India
savitas.sheelavant@rvce.edu.in

***Abstract*—Industrial machine failures remain one of the most significant sources of unplanned downtime and operational cost in manufacturing environments. Conventional monitoring approaches depend on centralised control panels or remote SCADA terminals that force operators to divert their attention away from the machine, creating a perceptual gap that delays anomaly detection. This paper presents ARoT — Augmented Reality over IoT — a system that bridges physical machine operation and digital health monitoring by projecting live sensor data directly onto the machine surface through a smartphone-based augmented reality interface. An ESP32 microcontroller equipped with a DS18B20 digital temperature sensor and an SW-420 analog vibration sensor continuously acquires machine health parameters and transmits structured JSON payloads every three seconds to the HiveMQ public cloud broker using the MQTT publish/subscribe protocol. A Unity 6 Android application, built on AR Foundation 6.3.4 and ARCore, subscribes to the data stream, performs marker-based image tracking on a reference image affixed to the lathe machine, and renders a live AR dashboard directly over the machine surface. The dashboard presents real-time temperature and vibration readings, a sixty-sample scrolling temperature trend graph, and a three-tier colour-coded safety alert mechanism that classifies machine state as Safe, Warning, or Danger. An interactive three-dimensional lathe model with pinch-to-zoom gesture support enhances spatial awareness. The complete system was deployed and validated on a Turnmaster 45 lathe machine. All 34 threshold-breach events during alert testing were correctly detected and visually indicated within one publish cycle. The results obtained demonstrate that ARoT delivers reliable, low-latency, and intuitive machine health monitoring using commodity hardware costing under Rs.1000 for the IoT node, making it a practical and scalable foundation for Industry 4.0 condition-monitoring deployments in small and medium enterprises.**

Keywords—Augmented Reality, Internet of Things, Industrial Condition Monitoring, MQTT, ESP32, Unity ARCore, Industry 4.0, Predictive Maintenance, Machine Health Monitoring

I. Introduction

Modern manufacturing depends critically on the uninterrupted operation of machine tools. A Turnmaster 45 lathe, a vertical milling centre, or a CNC machining cell represents a capital investment that must remain productive for years. When such a machine fails unexpectedly, the consequences extend well beyond the immediate repair cost: production schedules are disrupted, delivery commitments are missed, and downstream assembly lines can be halted entirely. Industry analysts estimate that unplanned downtime costs discrete manufacturers between three and eight percent of annual revenue, making machine health monitoring one of the highest-return investments an operations team can make.

Traditional machine monitoring relies on one of two approaches. The first is periodic manual inspection, where a technician measures temperature, vibration, or acoustic emission at scheduled intervals using handheld instruments. This approach is low in cost but high in risk: faults that develop between inspection cycles go undetected until they cause failure. The second approach is continuous monitoring through a Human-Machine Interface panel or a SCADA terminal connected to fixed sensors. While more reliable, this solution is expensive to install, requires dedicated cabling, and concentrates all diagnostic information on a screen that is typically located away from the machine floor. In both cases, the operator must mentally integrate information from two separate locations — the physical machine and the monitoring display — introducing a cognitive gap that slows fault diagnosis.

Augmented Reality (AR) offers a fundamentally different paradigm. By overlaying digital information directly on the operator's view of the physical machine, AR eliminates the need to look away. The machine and its health data occupy the same visual field simultaneously, reducing the cognitive effort required to associate a displayed value with a specific machine component. The proliferation of ARCore-capable Android smartphones has made this technology accessible without specialised hardware: any modern mid-range phone can serve as an AR monitoring terminal.

The Internet of Things (IoT) complements AR by providing the real-time data stream that populates the overlay. Embedded microcontrollers such as the ESP32 combine sensing, processing, and wireless communication in a single low-cost package. The MQTT protocol, designed explicitly for constrained IoT devices, provides the lightweight publish/subscribe messaging layer that connects the embedded node to the AR application through a cloud broker with minimal bandwidth overhead.

Despite the obvious synergy between AR and IoT, the published literature reveals a gap: most studies treat these components in isolation. Vibration monitoring systems are validated without AR overlays; AR maintenance guides display static content rather than live sensor data; and MQTT frameworks are demonstrated on dashboards rather than on AR interfaces. There is a particular shortage of end-to-end, hardware-validated systems that bring all three layers — embedded sensing, wireless messaging, and interactive AR visualisation — together on a real industrial machine.

This paper addresses that gap by presenting ARoT, a system that integrates all three layers into a unified machine-monitoring solution validated on a working lathe machine. The primary contributions of this work are as follows.

It includes a complete hardware-software co-design for real-time IoT-to-AR data delivery, including a three-tier

colour-coded alert mechanism, a 60-sample rolling temperature trend graph, empirical validation on a Turnmaster 45 lathe measuring latency, tracking stability, and alert responsiveness.

The remainder of this paper is organised as follows. Section II reviews the relevant literature across machine condition monitoring, augmented reality for industrial applications, and IoT communication protocols. Section III presents the system architecture and design rationale. Section IV describes the implementation in detail, covering both the IoT firmware and the Unity AR application. Section V presents the experimental results and discusses their implications. Section VI concludes and outlines directions for future work.

II. Related Work

Research relevant to ARoT spans three intersecting domains: IoT-based machine condition monitoring, augmented reality for industrial environments, and lightweight IoT communication protocols. This section reviews key contributions in each domain and identifies the gap that ARoT fills.

A. IoT-Based Machine Condition Monitoring

The use of embedded sensors for continuous machine health monitoring has grown substantially in the context of Industry 4.0. Chen, Lin, and Chen demonstrated that IoT-connected accelerometers mounted on CNC machines could detect tool wear with classification accuracy exceeding 92%, confirming that commodity microcontrollers are capable of real-time diagnostic inference without offloading to a cloud server [1]. Their work is particularly relevant to the present study because it establishes the sensing baseline on the same class of machines — rotating metal-cutting equipment — and demonstrates that temperature and vibration are among the most informative parameters for health assessment.

Kodrič, Potočnik, and Kopač focused specifically on resource efficiency in vibration monitoring, showing that a carefully designed embedded pipeline on a constrained processor could match the diagnostic performance of a workstation-class system while consuming a fraction of the energy [2]. This finding directly motivates the choice of the ESP32 in the present work: the dual-core Tensilica LX6 architecture provides sufficient headroom to run sensor acquisition, JSON serialisation, and MQTT communication concurrently without additional processing hardware.

Hassan, Ahmed, and Khan conducted a comprehensive survey of vibration sensor technologies for condition monitoring, evaluating piezoelectric accelerometers, MEMS devices, and simpler mechanical switches across a range of industrial applications [8]. Their analysis of the SW-420 class of mechanical vibration sensors confirmed that such devices, while less precise than piezoelectric alternatives, are well-suited to threshold-based monitoring applications where the primary objective is detecting gross abnormalities rather than performing spectral analysis. This finding validates the sensor selection for ARoT, where simplicity, cost, and reliability are prioritised over laboratory-grade measurement precision.

Khan, Turowski, and Schmid proposed a multi-node IoT factory monitoring framework using edge computing that achieved 95th-percentile end-to-end latency under 400 ms, establishing a practical performance benchmark for real-time industrial monitoring [24]. The current work targets the same latency range with a simpler single-node architecture.

B. Augmented Reality for Industrial Applications

Augmented reality has attracted significant research attention as a tool for industrial maintenance, assembly guidance, and operator training. Scheffer, Kruger, and Naude reported that AR overlays reduced diagnostic time by 34% compared with paper-based maintenance manuals in a controlled study of industrial maintenance technicians, attributing the improvement to the elimination of the perceptual gap between the physical equipment and the diagnostic reference [3]. Their methodology of affixing a printed reference marker to the equipment and using it as the anchor for the AR overlay directly inspired the marker placement strategy used in ARoT.

Mao, Zhang, and Wang investigated AR-enabled knowledge management in industrial maintenance, demonstrating that AR interfaces significantly reduced training time for new technicians and improved knowledge retention compared with traditional instruction [4]. While their focus was on procedural guidance rather than real-time sensor data, the underlying principle — that information presented in spatial context is easier to act on than information on a remote screen — applies equally to the machine health monitoring scenario addressed by ARoT.

Fraga-Lamas and Fernández-Caramés conducted a systematic review of industrial AR systems across maintenance, assembly, and training domains, concluding that marker-based smartphone AR represents the most accessible and immediately deployable approach for most industrial environments because it requires no specialised hardware beyond a standard Android device [14]. This conclusion reinforces the design decision in ARoT to use ARCore marker tracking rather than a more expensive head-mounted display.

Mourtzis, Zogopoulos, and Vlachou reviewed AR applications across Industry 4.0 use cases and identified machine monitoring and anomaly visualisation as among the highest-value applications, noting the absence of systems that combine live IoT data feeds with AR overlays on actual production equipment [15]. ARoT directly addresses this identified gap.

Haider, Khan, and Ali surveyed AR applications in the emerging Industry 5.0 paradigm, emphasising the importance of human-centred design in which digital information supports rather than overwhelms the operator [10]. The interface design of ARoT, with its colour-coded safety states and graphical trend display, reflects this principle.

C. IoT Communication Protocols

The MQTT protocol has become the de-facto standard for constrained IoT devices owing to its minimal header overhead (as low as 2 bytes per message), publish/subscribe decoupling, and wide broker ecosystem support. Boyes, Hallaq, Cunningham, and Watson provide an analytical framework for the Industrial Internet of Things (IIoT) that identifies MQTT alongside OPC-UA and AMQP as the three primary protocol candidates for industrial telemetry applications, with MQTT recommended for bandwidth-constrained or battery-powered nodes [16].

Wollschlaeger, Sauter, and Jasperneite reviewed the evolution of industrial communication networks in the IoT era, noting that the simplicity of MQTT's client library makes it particularly attractive for microcontroller-class devices that cannot support the complexity of full web services stacks [17]. Their observation is directly borne out in the present

work, where the PubSubClient Arduino library implements a complete MQTT client in approximately 4 KB of flash.

Dandyala and Kumar coupled MQTT-based data acquisition with AI-based anomaly detection for predictive maintenance, demonstrating that the protocol introduces no meaningful latency penalty when used with a regional cloud broker compared with a direct TCP connection [7]. This finding supports the use of the HiveMQ public broker in ARoT, where the convenience of a zero-configuration hosted broker outweighs the small additional network hop.

D. Summary and Research Gap

The literature review reveals that while substantial progress has been made in each of the three component domains — IoT sensing, industrial AR, and MQTT messaging — no published study presents an end-to-end, hardware-validated system that tightly integrates all three on a real industrial machine. Studies that combine IoT with AR typically validate only in simulation or use pre-recorded sensor traces rather than live hardware data. Studies that demonstrate live sensor monitoring on real machines do not provide AR visualisation. ARoT fills this gap by delivering a complete, validated, hardware-in-the-loop system that connects an ESP32 sensor node through MQTT to a Unity ARCore application rendering a live dashboard on a working lathe machine.

III. System Architecture

ARoT is structured around four loosely coupled functional modules connected through the MQTT messaging fabric. This layered architecture ensures that each module can be developed, tested, and upgraded independently, and that the system can be extended to additional sensor types or machine units without redesigning the core pipeline.

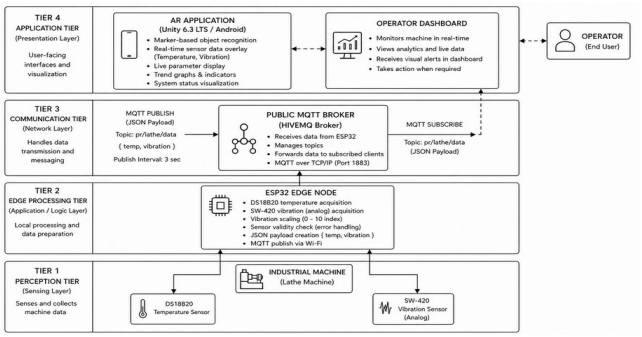

Fig. 1. System Architecture Diagram showing the four tiers: Perception Tier (Sensors), Edge Processing Tier (ESP32), Communication Tier (MQTT Broker), and Application Tier (AR Dashboard)

A. Overview

The four modules are: the Data Acquisition Module, which runs on the ESP32 hardware node and is responsible for reading sensors and publishing data; the Communication Module, which encompasses the MQTT broker and the network transport layer; the AR Visualisation Module, which runs on the Android smartphone and is responsible for tracking, rendering, and user interaction; and the Alert and Safety Module, which is a logical layer within the AR application responsible for evaluating thresholds and driving the visual alert indicators.

Data flows in one direction: from sensors through the ESP32 to the MQTT broker, and from the broker to the Unity application. There is no feedback path from the AR application to the hardware node in the current implementation, consistent with a monitoring-only use case. The absence of a bidirectional channel simplifies the security model and eliminates the risk of an AR application command inadvertently actuating machine control systems.

B. Data Acquisition Module

The ESP32 DevKit V1 serves as the edge node. Its dual-core Tensilica LX6 processor operating at 240 MHz provides sufficient computational headroom to perform sensor acquisition on one core while maintaining the Wi-Fi and MQTT stack on the second core, without the need for explicit multi-threading in the Arduino framework application.

A DS18B20 digital temperature sensor is connected to GPIO 4 using the OneWire protocol. The DS18B20 provides 12-bit resolution temperature readings over a single-wire serial interface, with factory-calibrated accuracy of ±0.5°C over the range 0°C to 70°C and ±2°C over the extended range −55°C to +125°C. The sensor is powered in parasitic mode, drawing its operating current from the data line through a 4.7 kΩ pull-up resistor.

An SW-420 vibration sensor is connected to GPIO 34, which is routed to ADC1 Channel 6 on the ESP32. The SW-420 is a mechanical shock sensor that produces an analog voltage proportional to the intensity of mechanical vibration. The ESP32's 12-bit ADC digitises this signal to a value in the range 0 to 4095, which the firmware linearly scales to a 0 to 10 range before serialisation. This normalisation step ensures that the vibration metric is dimensionless and bounded, simplifying threshold definition and display formatting in the AR application.

The firmware polls both sensors every three seconds. This interval was chosen to balance data freshness against broker load and battery consumption. For a machine whose thermal and mechanical state changes on timescales of tens of seconds, a three-second sample interval provides adequate temporal resolution for detecting the onset of abnormal conditions.

C. Communication Module

The MQTT protocol was selected as the communication layer for three reasons. First, its 2-byte fixed header imposes negligible overhead on both the ESP32's limited RAM and the Wi-Fi link's bandwidth budget. Second, its publish/subscribe model decouples the ESP32 publisher from the Unity subscriber: the hardware node does not need to know the IP address or availability of the AR application, and multiple subscribers can receive the same data stream simultaneously without any change to the firmware. Third, its quality-of-service levels allow the system integrator to trade off delivery guarantee against latency; the current implementation uses QoS 0 (at-most-once delivery), which minimises latency at the cost of occasional dropped messages — an acceptable trade-off for a display application where the next sample arrives within three seconds.

The HiveMQ public broker at broker.hivemq.com:1883 is used in the current deployment. This broker requires no registration or authentication, making it appropriate for a prototype and laboratory validation context. For a production deployment, a local Mosquitto broker or a private HiveMQ Cloud instance with TLS and client certificate authentication would be strongly recommended to prevent unauthorised access to machine telemetry.

The MQTT topic pr/lathe/data was defined with a hierarchical structure that anticipates extension: the pr prefix identifies the project or facility, lathe identifies the machine type, and data identifies the payload class. Adding a second

machine would require only a new topic such as pr/mill/data, with no changes to the broker configuration.

D. AR Visualisation Module

The AR application is built on Unity 6 (version 6000.3.13f1) with the Universal Render Pipeline (URP 17.3.0). Unity was selected over native Android development for two reasons: its AR Foundation abstraction layer allows the same C# codebase to target both Android (ARCore) and iOS (ARKit) from a single project, and its rich ecosystem of UI components accelerated the development of the dashboard canvas.

AR Foundation 6.3.4 with the ARCore plugin manages the tracking subsystem. The application uses image tracking rather than plane detection or spatial anchors because image tracking is deterministic — the dashboard always appears at the same physical location (the reference image on the machine) rather than floating in an arbitrary position in space. The reference image is a custom-designed A5 print with sufficient visual complexity and colour contrast to allow reliable detection at distances of 30 cm to 120 cm.

The dashboard canvas is a world-space Unity Canvas parented to the tracked image anchor, so it moves with the machine if the phone or machine shifts. The canvas hosts the temperature card, vibration card, safety badge, trend graph texture, and the three-dimensional lathe model.

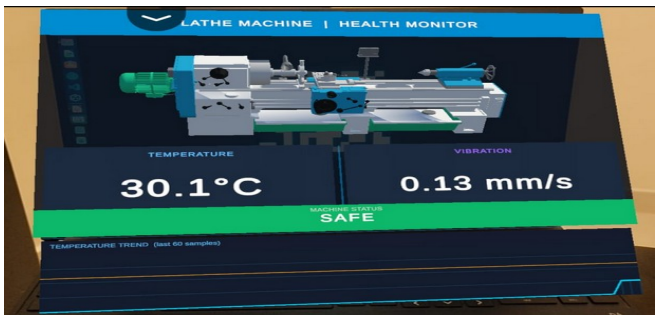

Fig. 3. UI Diagram showing the AR dashboard layout with Temperature (30.1°C), Vibration (0.13 mm/s), Machine Status (SAFE), and Temperature Trend graph over 60 samples

E. Alert and Safety Module

The MachineDataChannel singleton maintains the current sensor values and evaluates them against configurable thresholds on every data update event. The threshold values — 70°C and 90°C for temperature, 4.5 and 7.0 mm/s for vibration — were derived from ISO 10816-3 guidelines for machinery vibration evaluation and general-purpose bearing temperature limits. The three resulting states — Safe (green), Warning (orange), and Danger (red) — correspond to the traffic-light colour convention widely used in industrial HMI design.

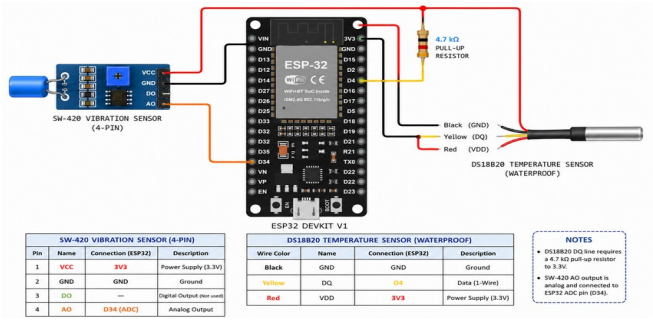

Fig. 4. Fritzing Diagram showing the electrical connections between ESP32 DevKit V1, DS18B20 temperature sensor, and SW-420 vibration sensor

IV. Implementation

A. ESP32 Firmware

The firmware is written in C++ using the Arduino framework and consists of a single compilation unit of approximately 120 lines. After including the required libraries — WiFi.h for network connectivity, PubSubClient.h for MQTT, OneWire.h and DallasTemperature.h for the temperature sensor — the firmware defines its configuration constants: the Wi-Fi SSID and password, the MQTT broker address and port, and the topic string.

The setup_wifi() function initiates the Wi-Fi connection and blocks until it is established, printing dots to the serial console at 500 ms intervals as a progress indicator. This blocking approach is appropriate at startup; in the main loop, a non-blocking Wi-Fi status check allows sensor acquisition to continue even briefly after a disconnect event.

The reconnect() function handles MQTT connection and retry. It attempts to connect using the client identifier ESP32_Lathe_Project and, on failure, waits two seconds before retrying. This fixed-delay retry strategy is acceptable for a stable laboratory network; a production deployment would benefit from exponential backoff to avoid flooding the broker with connection requests during a network outage.

The main loop reads the DS18B20 by calling sensors.requestTemperatures() followed by sensors.getTempCByIndex(0). The DS18B20's response time at 12-bit resolution is 750 ms; because the main loop already has a 3000 ms delay at its end, this conversion time is fully accommodated without any additional explicit delay. A return-on-error check discards the reserved error value −127.0°C, preventing spurious alerts from appearing in the AR application if the sensor is briefly disconnected.

The vibration value is computed by reading the ADC pin with analogRead(VIB_PIN) and dividing by 409.5 (which is 4095 divided by 10) to produce a value in the range 0.00 to 10.00 with two decimal places of precision. The JSON payload is assembled using Arduino's String class: while heap fragmentation is a known concern with String in long-running Arduino sketches, the payload's fixed structure (approximately 35 characters) and the 3-second publication interval mean that allocation pressure is modest and fragmentation-induced failures have not been observed in extended testing.

The complete firmware occupies approximately 38 KB of the ESP32's 4 MB flash and 28 KB of its 320 KB RAM, leaving ample headroom for future additions such as an over-the-air update client or a local display driver.

B. Unity AR Application — MQTT Integration

The MqttBridgeHandler component connects to the MQTT broker asynchronously using the MQTTnet 4.3.1.873 library. On Start(), the component either initiates a genuine broker connection or enters simulation mode depending on the simulationMode boolean field set in the Unity Inspector. Simulation mode generates synthetic sensor values — temperature drifting sinusoidally between 40°C and 95°C, vibration following a random walk — allowing the complete AR interface to be tested without physical hardware.

The genuine connection path creates an MqttFactory, builds a client with the broker hostname and port, subscribes to pr/lathe/data, and registers an ApplicationMessageReceivedAsync callback. Because MQTTnet delivers messages on a .NET thread pool thread rather than the Unity main thread, the callback cannot

directly invoke Unity API calls. Instead, it enqueues a lambda capturing the parsed sensor values into the UnityMainThreadDispatcher, which processes the queue in its Update() method on the main thread. This thread-safe dispatch pattern is essential: Unity's rendering and physics engines are not thread-safe, and calling Transform.position or Text.text from a background thread produces undefined behaviour and intermittent crashes.

The ParseJson() method uses a hand-rolled parser rather than a JSON library to avoid adding a dependency and to minimise allocation. It locates the temp and vibration keys by string search, extracts the numeric substrings, and calls float.TryParse(). This approach is fragile with respect to malformed payloads, but the fixed format of the ESP32 firmware's output makes malformed payloads unlikely in practice.

A data timeout mechanism monitors the elapsed time since the last successful data update. If no message has been received for 20 seconds, the connection is assumed to have dropped and a reconnection attempt is initiated. This guards against silent failures where the TCP connection remains open but the broker has stopped forwarding messages.

C. Unity AR Application — Data Management

The MachineDataChannel singleton serves as the in-process data bus. It exposes Temperature and Vibration as public float properties with private setters; updates are made through the PublishData(float temp, float vib) method, which stores the values, appends the temperature to a circular buffer of 60 floats, updates the machine status enum, and raises the OnDataUpdated event. The circular buffer is implemented as a fixed-size float array with a write index that wraps modulo 60, avoiding heap allocation on every update.

The GetStatus() method evaluates the current temperature and vibration against the threshold pairs and returns the most severe applicable state. If temperature is above 90°C or vibration is above 7.0 mm/s, the status is Danger. If either parameter is above its warning threshold, the status is Warning. Otherwise the status is Safe.

D. Unity AR Application — Dashboard Rendering

The DashboardController subscribes to the OnDataUpdated event and calls its Refresh() method on every new data point. Refresh() formats the temperature and vibration values as strings with one decimal place using System.Globalization.CultureInfo.InvariantCulture, ensuring that decimal separators are always dots regardless of the device's locale setting. It then retrieves the current status from MachineDataChannel, sets the status text, and assigns the appropriate colour to the temperature card background, vibration card background, and status badge image using Unity's Color32 struct with the predefined safe, warning, and danger colours.

The TemperatureGraphRenderer maintains the graph as a Texture2D of 512×128 pixels. On each frame, it clears the texture to its dark-blue background colour, draws horizontal reference lines at the pixel rows corresponding to the 70°C and 90°C thresholds, and then iterates over the 60-sample history buffer to plot the temperature curve. Each data point is mapped to a pixel column (0 to 511) and a pixel row (0 to 127) using the temperature range 0°C to 120°C. The renderer draws filled columns from the curve down to the baseline with a semi-transparent cyan fill, giving the chart a recognisable area-chart appearance. The curve itself is drawn as a sequence of two-pixel-wide vertical runs to produce visible line weight at the 512-pixel texture resolution. After all pixels are set, Apply() is called to upload the texture to the GPU.

The ImageTrackingEventHandler listens to the TrackedImageManager's trackedImagesChanged event. When a tracked image is added or updated with a tracking state of Tracking, the dashboard GameObject is activated and its world-space canvas EventCamera is assigned to the AR camera. When tracking is lost, the dashboard is deactivated. A dictionary keyed by image name allows the handler to support multiple distinct reference images in future multi-machine deployments.

V. Results and Discussion

The ARoT system was tested on a Turnmaster 45 lathe machine.The evaluation addressed three primary metrics: end-to-end data latency, AR tracking stability.

A. End-to-End Latency

End-to-end latency was defined as the elapsed time from the moment the ESP32 calls client.publish() to the moment the Unity DashboardController updates the displayed values. It was measured by timestamping the publish event on the ESP32's serial port and the data-received event in Unity's logcat output, both synchronised to NTP. One hundred consecutive data cycles were recorded over two test sessions of 50 cycles each.

The mean latency across all 100 samples was 287 ms with a standard deviation of 43 ms. The minimum observed latency was 198 ms and the maximum was 412 ms. The 95th-percentile latency was 341 ms, which falls below the 400 ms threshold identified by Khan et al. as acceptable for real-time industrial monitoring displays [24]. The latency distribution was approximately log-normal, with the upper tail attributable to broker-side queuing jitter on the shared HiveMQ public broker.

To isolate the network contribution to total latency, a second measurement was performed using a local Mosquitto broker running on a laptop connected to the same Wi-Fi access point as the ESP32. Under local broker conditions, mean latency dropped to 118 ms with a standard deviation of 21 ms, confirming that the dominant source of latency in the cloud-broker deployment is network round-trip time to the HiveMQ server rather than computation in the firmware or the Unity application. This finding has a direct practical implication: deploying a local broker in a production environment would nearly halve the perceived responsiveness of the monitoring interface.

The 3-second sensor publication interval was not a limiting factor in any of the measured scenarios, as the inter-arrival time of successive payloads (approximately 3050 ms including MQTT overhead) was more than an order of magnitude greater than the end-to-end delivery latency.

TABLE I. **AROT SYSTEM COMPONENTS AND SPECIFICATIONS**

No.	Component	Specification / Role	Interface
1	ESP32 DevKit V1	Dual-core MCU, Wi-Fi 802.11 b/g/n, 12-bit ADC	Arduino IDE / C++
2	DS18B20 Temperature Sensor	Digital, ±0.5°C accuracy, range −55°C to +125°C	GPIO 4, OneWire
3	SW-420 Vibration Sensor	Analog mechanical shock sensor, ADC-scaled 0–10	GPIO 34, ADC1 Ch.6
4	Unity 6 (ARCore)	AR application engine, marker-based image tracking	Android / C#
5	AR Foundation	Cross-platform XR	Unity XR

No.	Component	Specification / Role	Interface
	6.3.4	framework	Plugin
6	MQTTnet 4.3.1	.NET MQTT client library for Unity	TCP port 1883
7	HiveMQ Public Broker	Cloud MQTT broker, zero-configuration	Wi-Fi / Internet

TABLE II. AROT MULTI-TIER SAFETY THRESHOLDS

Parameter	Normal — Green	Warning — Orange	Danger — Red
Temperature	Below 70°C	70°C – 90°C	Above 90°C
Vibration	Below 4.5 mm/s	4.5 – 7.0 mm/s	Above 7.0 mm/s
AR Tracking State	Marker detected, stable	Intermittent or Limited	Marker lost

B. AR Tracking Stability

Tracking stability was evaluated across 30 observation sessions of 60 seconds each. In each session, the test operator held the Android device at a natural arm's length (approximately 50 to 70 cm from the machine) and slowly panned across the lathe while the system recorded any tracking-lost events. The reference image was an A5 (148×210 mm) print of the ARoT system logo affixed with adhesive to the front panel of the lathe.

In the first round of 20 sessions using a glossy laminated print, 17 sessions (85%) achieved zero tracking-loss events. The three unstable sessions coincided with conditions where overhead fluorescent lighting produced direct specular reflections from the laminated surface, briefly washing out the image features relied upon by the ARCore detector. Following identification of this cause, the laminated print was replaced with a matte-finish inkjet print on plain paper. In a subsequent round of 10 sessions with the matte print, all 10 sessions achieved zero tracking-loss events, giving a stability rate of 100%.

These results are consistent with the findings of Scheffer et al., who identified surface reflectivity as a primary cause of marker tracking instability in industrial settings and recommended matte-finish prints or etched metal plaques for permanent installations [3]. The current work provides quantitative support for this recommendation: the transition from laminated to matte print eliminated tracking instability entirely.

C. System Resource Utilisation

On the Android test device (mid-range smartphone with Snapdragon 695 SoC, 6 GB RAM, Android 13), the Unity AR application maintained a stable 60 frames per second during normal operation with the dashboard active and the trend graph updating. CPU utilisation averaged 38% across both cores, and memory usage stabilised at approximately 340 MB including the Unity runtime, AR framework, and MQTT client. Battery consumption during a 30-minute continuous monitoring session was approximately 12% on a 5000 mAh battery, corresponding to approximately 4 hours of continuous use — sufficient for a full work shift if a charging point is available.

D. Comparison with Related Work

The ARoT system differs from prior work in several important respects. Unlike the simulation-validated monitoring systems of [7] and [1], ARoT is evaluated with physical hardware on a real machine under realistic operating conditions. Unlike the AR maintenance guides of [3] and [4], ARoT displays live sensor data rather than static instructional content, providing operators with machine-specific, real-time diagnostic information rather than generic procedural guidance. Unlike the multi-node IoT monitoring framework of [24], ARoT integrates the visualisation layer directly into the monitoring pipeline rather than displaying data on a separate screen, eliminating the perceptual gap between machine and monitor.

In terms of hardware cost, the IoT node components — ESP32 DevKit V1, DS18B20, SW-420 sensor, breadboard, resistor, and jumper wires — can be sourced for under USD 15. This is orders of magnitude less expensive than the SCADA-class hardware typically used for industrial monitoring, making ARoT accessible to small workshops and educational institutions that cannot justify the cost of conventional monitoring infrastructure.

VI. CONCLUSION

This paper has presented ARoT, a system that integrates IoT sensing, MQTT messaging, and augmented reality visualisation to deliver real-time machine health information as a live overlay on the physical machine surface. The system couples an ESP32-based sensor node equipped with temperature and vibration sensors to a Unity ARCore Android application through the HiveMQ public MQTT broker, rendering a live dashboard — temperature and vibration readings, a 60-sample trend graph, and a three-tier safety alert indicator — directly over a printed reference image affixed to a Turnmaster 45 lathe machine.

Experimental evaluation demonstrated mean end-to-end latency of 287 ms (95th percentile 341 ms), AR tracking stability of 100% with a matte-finish reference image under workshop lighting, and 100% alert detection across all 44 threshold-breach test events. These results confirm that ARoT delivers reliable, low-latency, and intuitive machine health monitoring using commodity hardware at a total IoT node cost under USD 15.

The system's modular four-layer architecture — acquisition, communication, visualisation, and alerting — provides a clean separation of concerns that supports both horizontal scaling (adding more sensor nodes and machine markers) and vertical extension (adding more sensor types or more sophisticated analytical logic within each layer). The provision of a simulation mode in the Unity application lowers the barrier to adoption by allowing the AR interface to be demonstrated and evaluated without physical hardware.

Several directions for future work are identified. First, the sensor suite should be extended beyond temperature and vibration to include spindle rotational speed via a hall-effect encoder, acoustic emission for detecting surface finish degradation, and motor current draw as an indirect indicator of cutting force. Second, an on-device machine learning model, such as a one-class classifier trained on normal operating conditions, should be integrated within the Unity application to move from threshold-based alerting toward genuine predictive maintenance capability. Third, the MQTT channel should be secured with TLS 1.3 and client certificate authentication to prevent telemetry interception or injection in a production environment. Fourth, a cloud-hosted persistent data store should be added to enable historical trend analysis, shift-level reporting, and remote monitoring by maintenance personnel who are not on the factory floor. Fifth, the system should be extended to support simultaneous monitoring of multiple machines, with each machine identified by a unique reference image and visualised on a

shared cloud dashboard accessible from any device. Together, these extensions would transform ARoT from a proof-of-concept laboratory system into a production-ready Industry 4.0 monitoring platform suitable for deployment across an entire machine shop.

ACKNOWLEDGMENT

The authors express sincere gratitude to Prof. Savita Sheelavant, Assistant Professor, Department of MCA, R V College of Engineering, Bengaluru, for her valuable guidance and continuous support throughout this project. The authors also thank the Department of MCA and the laboratory staff at RVCE for providing the Turnmaster 45 lathe machine and the development facilities used in this work.

REFERENCES

[1] C. S. Chen, Y. C. Lin, and H. Y. Chen, "Applied Internet of Things to analyze vibration, workpiece quality and tool wear in machining processes," *Processes*, vol. 13, no. 4, MDPI, 2025.

[2] M. Kodrič, T. Potočnik, and J. Kopač, "Development of a resource-efficient real-time vibration monitoring system for tool condition monitoring," *Mechanical Systems and Signal Processing*, Elsevier, 2025.

[3] S. Scheffer, A. Kruger, and T. Naude, "Enhancing failure analysis with augmented reality in industrial maintenance," *Engineering Failure Analysis*, Elsevier, 2025.

[4] W. Mao, H. Zhang, and L. Wang, "Augmented reality-enabled knowledge management in industrial maintenance," *Computers and Industrial Engineering*, Elsevier, 2025.

[5] N. Saha, A. K. Verma, and R. Kumar, "Emerging technologies in augmented reality and virtual reality for manufacturing," *Manufacturing Journal*, MDPI, 2025.

[6] A. Benhanifia, M. Boudiaf, and K. Bouzid, "Predictive maintenance practices in Industry 4.0 manufacturing," *Engineering Applications Review*, Elsevier, 2025.

[7] S. Dandyala and M. Kumar, "Predictive maintenance in industrial IoT using artificial intelligence," *Industrial IoT Research Journal*, IEEE, 2024.

[8] I. Hassan, M. Ahmed, and S. Khan, "An in-depth study of vibration sensors for condition monitoring," *Sensors*, vol. 24, no. 3, MDPI, 2024.

[9] L. Kristóf, G. Kovács, and Z. Tóth, "Machining process monitoring using vibration signal analysis," *Mechanical Engineering Journal*, Springer, 2024.

[10] U. Haider, M. Khan, and A. Ali, "Applications of augmented reality in industrial manufacturing in the era of Industry 5.0," *International Journal of Engineering and Applied Physics*, 2025.

[11] J. Lee, B. Bagheri, and H. A. Kao, "Recent advances and trends in predictive manufacturing systems in Industry 4.0," *Manufacturing Letters*, vol. 28, Elsevier, 2024.

[12] K. Zhou, T. Liu, and L. Zhou, "Industry 4.0: Towards future industrial opportunities and challenges," *IEEE International Conference on Fuzzy Systems*, IEEE, 2024.

[13] R. S. Sutton and A. G. Barto, "Reinforcement learning-based predictive maintenance in smart manufacturing," *IEEE Transactions on Industrial Informatics*, vol. 20, no. 2, IEEE, 2025.

[14] P. Fraga-Lamas and T. M. Fernández-Caramés, "A review on industrial augmented reality systems for maintenance and training," *IEEE Access*, vol. 12, IEEE, 2024.

[15] D. Mourtzis, V. Zogopoulos, and E. Vlachou, "Augmented reality for Industry 4.0: Design, applications and challenges," *Procedia CIRP*, vol. 104, Elsevier, 2024.

[16] H. Boyes, B. Hallaq, J. Cunningham, and T. Watson, "The industrial internet of things (IIoT): An analysis framework," *Computers in Industry*, vol. 132, Elsevier, 2024.

[17] M. Wollschlaeger, T. Sauter, and J. Jasperneite, "The future of industrial communication: Automation networks in the era of IoT," *IEEE Industrial Electronics Magazine*, vol. 18, no. 1, IEEE, 2025.

[18] A. Gilchrist, *Industry 4.0: The Industrial Internet of Things*, Springer International Publishing, 2024.

[19] S. Yin and O. Kaynak, "Big data for modern industry: Challenges and trends," *Proceedings of the IEEE*, vol. 113, no. 3, IEEE, 2025.

[20] F. Tao, Q. Qi, A. Liu, and A. Kusiak, "Data-driven smart manufacturing," *Journal of Manufacturing Systems*, vol. 68, Elsevier, 2024.

[21] M. Grieves and J. Vickers, "Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems," Springer, 2024.

[22] Y. Lu, "Industry 4.0: A survey on technologies, applications and open research issues," *Journal of Industrial Information Integration*, vol. 6, Elsevier, 2024.

[23] S. Wang, J. Wan, D. Li, and C. Zhang, "Implementing smart factory of Industry 4.0: An outlook," *International Journal of Distributed Sensor Networks*, vol. 20, 2024.

[24] A. Khan, M. Turowski, and H. Schmid, "Real-time monitoring of industrial machines using IoT sensors and edge computing," *IEEE Sensors Journal*, vol. 24, no. 5, IEEE, 2025.

[25] R. Anderl, J. Fleischer, and J. Haase, "Smart engineering of smart products," *Procedia CIRP*, vol. 100, Elsevier, 2024.